

IoT Speed Camera & Remote-Controlled Car

Technical Project Report

Embedded Software for the Internet of Things
Group 14

Team Members

Name	Student ID
Luciani Stefano	243851
De Cao Andrea	244157
Boscardin Denise	243932
Heenatigala Devmin	242916

Academic Year 2025/2026

1. Project Overview

This project was developed as part of the Embedded Software for the Internet of Things course. It consists of two main subsystems: a gesture-controlled remote-controlled (RC) car, and a scaled-down speed camera system.

Together they form a complete IoT pipeline that spans hardware sensing, wireless communication, embedded control, and web-based data visualization.

The RC car is a custom-built four-wheeled vehicle assembled on a chassis with four 5V DC motors controlled by an L298N H-bridge driver, powered by an Arduino Uno R4 WiFi. It is operated through hand gestures detected by an MPU6050 gyroscope sensor mounted on a separate Arduino Nano controller unit.

The operator tilts their hand to steer and accelerate the vehicle, with movement commands transmitted wirelessly via nRF24L01 radio modules.

The speed camera, built around an ESP32 microcontroller, measures the vehicle speed using two VCNL4010 proximity sensors and compares it against a configurable threshold. Detection results are transmitted to a cloud-hosted Node.js web server and displayed in a real-time dashboard.

2. System Architecture

The system is composed of three hardware units and one software layer:

2.1 Arduino Nano — Gesture Controller

The controller unit is built around an Arduino Nano paired with an MPU6050 IMU. The MPU6050's Digital Motion Processor (DMP) computes quaternion-based orientation data, which is converted to Yaw, Pitch, and Roll angles. Roll controls left/right steering, Pitch controls forward/backward movement. Angle values are constrained to the range $[-50^\circ, +50^\circ]$ and mapped to a byte (0–254) for compact transmission over nRF24L01 at 250 kbps on channel 108 (outside standard Wi-Fi bands).

2.2 Arduino Uno R4 WiFi — Car Receiver

The car unit receives the nRF24L01 packets and uses differential drive logic to compute independent left and right motor speeds. The L298N H-bridge motor driver controls four DC motors. Speed for each side is derived from the received Roll and Pitch values, enabling

smooth curved trajectories. The radio operates in RX mode with automatic ACK to confirm packet reception.

2.3 ESP32 — Speed Camera Unit

The speed camera uses two VCNL4010 proximity sensors mounted 4 cm apart on separate I2C buses. A finite state machine (FSM) manages the detection pipeline: the system waits for the first sensor to trigger, records a microsecond timestamp, then waits for the second sensor. Speed is calculated as distance divided by elapsed time and compared against the 0.75 m/s threshold.

2.4 Node.js Web Server — Data Layer

The backend is a Node.js/Express server deployed on Render. It exposes three REST endpoints:

Endpoint	Method	Description
POST /salva	POST	Receives speed and overspeed flag from the ESP32, appends a timestamped row to a CSV log file with an auto-incremented test case ID.
GET /dati	GET	Returns the full speed log as a JSON array for table display in the dashboard.
GET /data	GET	Returns only the latest reading (speed + sensor flag) for real-time display.

Data is persisted in a CSV file (speed-log.csv) with the columns: Test-Case-ID, Speed, Sensor-Triggered, Timestamp.

The server also serves the frontend static files from the public/ directory.

3. Firmware Description

3.1 Speed Camera FSM (fsm_velox.ino)

The speed camera firmware implements a 4-state FSM:

State	Description
WAIT	Idle state. Updates the OLED display and transitions to IR1_STATE.
IR1_STATE	Polls BAR1 proximity. When proximity exceeds the calibrated threshold, records tempo1 (microseconds) and transitions to IR2_STATE.

IR2_STATE	Polls BAR2 proximity. Enforces a 3-second timeout: if the vehicle does not pass within the limit, resets to WAIT. On trigger, records tempo2.
COMPUTATION	Calculates speed = (4 cm / delta_t) converted to m/s. Sets overSpeed flag, updates OLED, saves to flash, and sends an HTTPS POST to the server.

Sensor calibration is performed at startup: 20 proximity readings are averaged per sensor, and a fixed offset (SOGLIA_OFFSET = 50) is added to define the detection threshold, accounting for ambient IR noise.

3.2 Gesture Transmitter (nRF24L01_TX.ino)

The transmitter firmware runs on Arduino Nano. The MPU6050 DMP is initialized with 6-cycle accelerometer and gyroscope calibration for accurate zero-gravity offset. In the main loop, each FIFO packet from the DMP is read, quaternions are extracted, gravity is computed, and Euler angles (Yaw/Pitch/Roll) are derived. Angles are clamped and mapped to bytes, packed into a 3-byte DataPacket struct, and transmitted via radio.write(). The nRF24L01 is configured with 5 retries at 250 us intervals to handle packet loss.

3.3 Car Receiver (nRF24L01_RX.ino)

The receiver firmware runs on Arduino Uno R4 WiFi. The radio is set to RX mode on the same channel and address as the transmitter. On each received packet, the movimento() function computes differential motor speeds: moveX from Roll (left/right bias) and moveY from Pitch (forward/backward velocity). The constrain() function ensures PWM values stay within the 0–255 range, enabling smooth turning at variable speeds.

4. Web Server & Dashboard

4.1 Backend (server.js)

The server is implemented with Express.js and reads/writes a local CSV file for data persistence. On startup, it reads the existing CSV to restore the currentTestCaselId counter, ensuring unique IDs across server restarts. The POST /salva endpoint includes a basic duplicate-detection check comparing the last CSV row against the incoming data before appending.

4.2 Frontend Website (index.html)

The project website serves as the main entry point, providing a project overview, an image slider with hardware schematics, a system architecture section, key features, a how-it-works walkthrough, an embedded YouTube demonstration, and links to firmware and documentation.

4.3 Real-Time Dashboard (DashBoard.html)

The dashboard polls the GET /data endpoint at regular intervals to display the latest vehicle speed and overspeed status. It also fetches the full log from GET /dati to populate a historical data table. The dashboard is accessible at iot-project-group-14.onrender.com.

5. Communication Protocol

Link	Protocol	Parameters	Notes
Controller > Car	nRF24L01 (SPI)	250 kbps, channel 108, PA_MIN	3-byte struct: Roll, Pitch, Yaw (0–254 each)
ESP32 > Server	HTTPS / REST	POST /salva, JSON body	WiFiClientSecure, certificate check disabled
Server > Dashboard	HTTP / REST	GET /data, GET /dati	JSON response, polled by frontend JS
I2C (ESP32)	I2C @ 100 kHz	Two buses: pins 21/22 and 18/19	VCNL4010 on both buses, OLED on bus 1
I2C (Nano)	I2C @ 400 kHz	Fast Mode, 3 ms timeout	MPU6050 DMP, EMI protection via timeout

6. Hardware Components

6.1 RC Car

Component	Purpose
Arduino Uno R4 WiFi	Main car controller, motor driver interface
Arduino Nano	Gesture controller unit
MPU6050	Gyroscope/accelerometer for gesture detection
nRF24L01+ PA LNA (x2)	Wireless communication between controller and car
L298N Motor Driver	Dual H-bridge for 4 DC motors
4x 5V DC Motors + Tires	Propulsion system

Power Bank 5000/10400 mAh	Power supply for electronics and motors
---------------------------	---

6.2 Speed Camera

Component	Purpose
ESP32	Main speed camera controller, WiFi connectivity
VCNL4010 (x2)	Proximity sensors for speed measurement (4 cm apart)
SSD1306 OLED Display	Real-time status and speed display
Breadboard & Jumper Wires	Circuit connections

7. Software Requirements

Tool / Library	Version	Purpose
Arduino IDE	2.x	Firmware development and upload
RF24	Latest	nRF24L01 radio communication
MPU6050 / I2Cdev	Latest	Gyroscope DMP processing
Adafruit VCNL4010	Latest	Proximity sensor driver
Adafruit SSD1306	Latest	OLED display driver
Node.js / Express	LTS	Web server backend
Render	—	Cloud deployment platform

8. Team Members & Contributions

Luciani Stefano

Responsible for the hardware design, component wiring and the development of the control and communication software for RC car.

Responsible for the development of the finite state machine of the velox and its hardware design.

De Cao Andrea

Responsible for the overall project structure and architecture. Co-developed the control and communication software for the RC car, developed a Node.js backend exposing a REST API to receive and serve real-time sensor data, and a frontend Dashboard displaying live vehicle data. Handled the frontend–backend integration and deployment on Render.

Boscardin Denise

Responsible for the speed camera firmware, 3D printing of physical enclosures and mechanical parts, README documentation, and video editing of the project demonstration.

Heenatigala Devmin

Responsible for the speed camera code and code polishing.

9. Deployment & Setup

9.1 Web Server

To run the web server locally:

```
cd src/web_server
npm install
npm start
# Server starts at http://localhost:3000
```

The server is also deployed on Render at:

```
https://iot-project-group-14.onrender.com/
```

9.2 Firmware Upload Order

1. Upload fsm_velox.ino to ESP32 (speed camera).
2. Upload nRF24L01_RX.ino to Arduino Uno R4 WiFi (car receiver).
3. Upload nRF24L01_TX.ino to Arduino Nano (gesture controller).
4. Keep the MPU6050 flat during Nano startup for calibration.
5. Power on the car, then the speed camera.
6. Open the web dashboard to monitor speed data.

10. Links & Resources

Resource	URL
Web Dashboard	https://iot-project-group-14.onrender.com/
GitHub Repository	https://github.com/AndreaDeCao/IoT_project
YouTube Demo	https://www.youtube.com/watch?v=iuvEgluySTU